

Распределённые нейронные сети на микроконтроллерах ESP32: аналитический и практический отчёт для ВКР

Executive summary

Тема диплома «Распределённая сеть нейронных сетей на микроконтроллерах (ESP32)» находится на пересечении TinyML, сетевых протоколов IoT и системной оптимизации (память/энергия/задержка). Главный вывод современной литературы и практики: **в распределённом инференсе узким местом почти всегда оказывается не “сколько MAC/s у ядра”, а стоимость коммуникации (объём промежуточных тензоров, jitter, потери) и непредсказуемость системного стека Wi-Fi/RTOS**. Поэтому сильная дипломная работа должна формулировать задачу не как «запустить сеть на нескольких ESP32», а как **многокритериальную оптимизацию точности, задержки (особенно p95/p99), трафика и энергии с учётом сетевого состояния**.

1

Практически наиболее реалистичная и “защитимая” стратегия для ESP32-кластера следующая:

- 1) **Опора на ESP32-S3 как на основной вычислительный узел** (SIMD-инструкции, встроенный USB Serial/JTAG, достаточный SRAM и хорошая поддержка в экосистеме Espressif-TinyML). 2
- 2) **Кооперация узлов не на уровне “сырой активации ранних слоёв”, а на уровне компактных токенов**: логиты/латенты фиксированного бюджета (десятки байт), confidence и метрики качества канала (RSSI/SNR). Для ESP-NOW это особенно важно, т.к. максимальная полезная нагрузка ограничена (v1 до 250 B, v2 до 1470 B), а колбэки выполняются в high-priority Wi-Fi task и не терпят тяжёлой обработки. 3
- 3) **Адаптивная маршрутизация (routing policy)**: “лёгкие” примеры проходят дешёвым быстрым маршрутом (локально или через fusion логитов), “трудные” — активируют второй этап (second-stage) на более мощном узле или по другому маршруту. Это концептуально близко к идеям *early-exit* и *co-inference* (Edgent), но переносится на MCU-to-MCU. 4
- 4) Научная новизна диплома может быть сформулирована как **communication-aware обучение/дистилляция и latency-aware routing**, а не “просто инженерка”. 5

В предоставленном вами внутреннем проектном документе уже сформулирована очень удачная рамка “распределённого нейрографа / neural fabric” и критика наивного layer-split (передачи крупных активаций). Это хорошая заготовка: её стоит усилить ссылками на *NoNN* (communication-aware декомпозиция), *BottleNet/BottleFit* (обучаемые bottleneck-представления для split-вычислений), а также добавить строгий экспериментальный протокол (MLPerf Tiny-подобная дисциплина). 6

Ниже “не указано” (и в отчёте отмечено как неопределённость): **точный дедлайн, доступный бюджет, конкретная предметная задача/датасет** (аудио/IMU/аномалии/vision). Эти параметры влияют на выбор архитектуры и стека.

Технический обзор ESP32 для чайника

Что такое ESP32 “в системном смысле”

ESP32-семейство — это SoC-микроконтроллеры со встроенным радио (Wi-Fi 2.4 GHz и, в зависимости от чипа, Bluetooth), периферией и контроллерами памяти. Ключевое отличие от “маленького компьютера”: **нет виртуальной памяти, нет swar, крайне ограничен быстрый SRAM, и любая лишняя копия буфера быстро превращается в задержки и потери real-time дедлайнов.** ⁷

Неприятная правда для распределённого ML: даже если один слой считается быстро, итоговая end-to-end задержка складывается из: (1) сенсинга, (2) препроцессинга, (3) инференса, (4) очередей RTOS, (5) передачи по радио, (6) синхронизации/fusion/решения — и очень часто “ломается” пунктах (4) и (5). ⁸

Архитектура CPU и вычислительный бюджет

Классический ESP32: - CPU: Xtensa LX6, 32-бит, 1–2 ядра, до 240 MHz. ⁹

- Типичная роль в проектах: IoT-контроллер + радио + умеренная TinyML (аудио, простая классификация), но без большого запаса по вычислениям и памяти. ¹⁰

ESP32-S2: - CPU: Xtensa LX7, 1 ядро, до 240 MHz. ¹¹

- Фокус: Wi-Fi-устройства (в S2 нет классического Bluetooth; по линейке Espressif S2 позиционируется как Wi-Fi-SoC). ¹¹

ESP32-C3: - CPU: 32-бит RISC-V, до 160 MHz. ¹²

- Память по продуктовой странице: порядка 400 KB internal RAM (плюс внешняя flash). ¹²

- Роль: “лёгкий” узел для сенсинга/предиктов/маршрутизации, но слабее для тяжёлой нейросети. ¹³

ESP32-S3 (наиболее предпочтителен для диплома): - CPU: Xtensa LX7, 2 ядра, до 240 MHz. ¹⁴

- Важное для NN: *128-бит шина данных и выделенные SIMD-инструкции* (Это именно то, что “цепляется” оптимизированными kernels, напр. ESP-NN). ¹⁵

- Отладка/инженерия: встроенный USB Serial/JTAG позволяет дебажить без внешнего программатора (сильно снижает риск проекта). ¹⁶

Память: SRAM, PSRAM и Flash — как думать правильно

Для TinyML на ESP32 важно разделять три уровня:

Flash (внешняя SPI/QSPI/OSPI, часто 4–16 MB на платах):

- Хранит прошивку, константы, модель (веса) и ресурсы.

- Доступ к данным часто идёт через кэш/мэппинг, но это всё равно “не SRAM” по латентности и пропускной способности. ¹⁷

Внутренняя SRAM (самая “золотая” память): - ESP32: суммарно порядка 520 KB SRAM + RTC SRAM. ⁹

- ESP32-S3: 512 KB SRAM + 16 KB RTC SRAM + отдельные RTC FAST/SLOW области (8 KB + 8 KB), которые могут сохраняться в Deep-sleep. ¹⁸

- Именно здесь должны жить: arena TFLite Micro (или аналоги), горячие буферы активаций, очереди real-time и структуры планировщика. ¹⁹

PSRAM (external RAM, SPI-PSRAM, часто 2-16 MB): - Даёт “вместимость”, но **может ухудшать предсказуемость задержек**: доступ через SPI и кэш, другие требования к DMA и выравниванию, и главное — внешняя память обычно не подходит для самых горячих участков инференса. ²⁰

Практический принцип: **PSRAM — для “холодного”** (логи, вторичные буферы, большой ring buffer, не-RT структуры), **SRAM — для “горячего” и детерминированного пайплайна**. ²¹

Периферия, важная именно для диплома по распределённому ML

- **I2S** (микрофон/аудио) — основной вход для KWS/акустических событий. У ESP32 есть два I2S. ⁹
- **Camera interface** — у ESP32-S3 есть камера-интерфейс (важно, если вы захотите дешёвую vision-демонстрацию на маленьком разрешении). ²²
- **ADC/DAC, touch, PWM, UART/SPI/I2C** — для IMU/датчиков вибрации/газовых датчиков и т.п. ²³

Wi-Fi / Bluetooth: throughput, latency, power-save

Wi-Fi возможности (пример ESP32-S3): - IEEE 802.11 b/g/n, 20/40 MHz, 1T1R, PHY-rate до 150 Mbps. ²²

- В реальности throughput зависит от стека и условий. Espressif публикует лабораторные ориентиры для ESP32: например, UDP RX/TX порядка десятков Mbit/s “over the air”, в shield box больше. ²⁴

Power-save и “скрытая задержка”: - В ESP-IDF задокументировано, что при включённом Modem-sleep задержка приёма может быть сопоставима с DTIM cycle или listening interval; отключение power-save уменьшает задержку приёма, но увеличивает потребление. ²⁵

Bluetooth и BLE: - ESP32 (classic): поддерживает Bluetooth v4.2 BR/EDR + BLE (dual mode). ²⁶
- ESP32-S3: Bluetooth 5 (LE) (то есть BLE-ориентированность). ²⁷

Энергопотребление и частоты — что важно для метрик диплома

Для диплома нужно мыслить не “мА в даташите”, а **энергией на инференс/на событие**, и распределять энергию на compute vs radio.

Ориентиры (даташиты): - ESP32: deep-sleep порядка 10 µA (в зависимости от режимов/домена питания). ⁹

- ESP32-S3: заявляются режимы Active/Modem-sleep/Light-sleep/Deep-sleep, и deep-sleep порядка единиц µA (типичные значения ~7-8 µA в таблицах low-power). ²⁸

Важное следствие: в распределённой системе радио-передачи и повторные передачи могут разрушить энергетический выигрыш от “распараллелили вычисления”, поэтому **в дипломе обязательно сравнивать energy/latency/accuracy вместе** (как в логике MLPerf Tiny). ²⁹

Типичные платы для стенда

Официальные dev-kits (удобны для воспроизводимости и ссылок в дипломе):

- ESP32-DevKitC (классический ESP32). ³⁰
- ESP32-S2-DevKitM-1. ³¹
- ESP32-C3-DevKitM-1. ³²
- ESP32-S3-DevKitC-1 (лучший “базовый кирпич” под ваш диплом). ³³

Инструменты разработки, отладка и прошивка

ESP-IDF (официальный фреймворк)

- Сборка/прошивка управляются `idf.py`, который является “фронтендом” к CMake/Ninja/esptool.py. ³⁴
- Мониторинг UART-логов: `idf.py monitor` (idf-monitor). ³⁵
- Разметка flash и OTA завязаны на partition table; типичные смещения: factory app часто начинается с 0x10000. ³⁶

Отладка - Для ESP32-S3 есть руководство по JTAG debugging в ESP-IDF, включая работу с OpenOCD/GDB. ³⁷

- Трассировка RTOS/событий: application-level tracing и интеграция с SEGGER SystemView. ³⁸

Arduino-экосистема

- Arduino-core для ESP32 поддерживается Espressif, есть официальная документация. ³⁹
- Для диплома Arduino удобен для быстрых прототипов, но ESP-IDF обычно лучше для строгих измерений и управления задачами/памятью/радио. ⁴⁰

MicroPython

- Порт MicroPython для ESP32 построен поверх ESP-IDF и работает как задача под FreeRTOS; удобно для обучения и быстрых экспериментов, но сложнее получить детерминированный real-time и минимальный overhead. ⁴¹

Фреймворки и библиотеки нейросетей на MCU

Карта экосистемы: “рантайм” против “ускорителей” против “конвертера”

Для MCU-TinyML пайплайн почти всегда состоит из: 1) обучения/подготовки модели на ПК, 2) компрессии/квантования, 3) конвертации в формат исполнения, 4) рантайма на MCU + набор операторов (kernels), 5) аппаратно-специфичных оптимизаций (SIMD/DSP/вендорные kernels).

⁴²

Основные кандидаты под ESP32

LiteRT / TensorFlow Lite for Microcontrollers (TFLM)

- Официальная линия Google для инференса на микроконтроллерах (интерпретатор + статическая memory arena). ⁴³
- Основная сильная сторона: зрелость, много примеров, хорошая поддержка INT8-квантования, широкое комьюнити. ⁴⁴

TFLM на Espressif (esp-tflite-micro)

- Espressif поддерживает репозиторий с примерами (hello_world, micro_speech, person_detection), интеграция с ESP-IDF. ⁴⁵

ESP-NN

- Библиотека оптимизированных NN-функций для чипов Espressif; прямо указано, что для ESP32-S3 есть assembly-версии, использующие vector/SIMD-инструкции, и что библиотека протестирована с TFLite Micro. ⁴⁶

ESP-DL

- Фреймворк Espressif для инференса (и сопутствующих операций) на ESP-чипах; в экосистеме также упоминается инструмент квантования ESP-PPQ (из ONNX/PyTorch/TensorFlow в формат ESP-DL). ⁴⁷

microTVM (Apache TVM для микроконтроллеров)

- Подход “компилируем модель в C/коды под target”, потенциально даёт лучшую оптимизацию под конкретную платформу, но требует дисциплины в сборке и поддержке операторов. ⁴⁸

TinyEngine / MCUNet-линия

- MCUNet (TinyNAS + TinyEngine) — показательная работа: на MCU выигрывает совместный дизайн модели и рантайма; TinyEngine снижает peak SRAM и ускоряет инференс относительно TFLM/CMSIS-NN в экспериментах авторов. ⁴⁹

- Для диплома: это эталон “как правильно думать про memory scheduling”, даже если вы не будете использовать TinyEngine напрямую. ⁵⁰

CMSIS-NN / uTensor

- CMSIS-NN — набор оптимизированных kernels под Cortex-M (Arm), полезен как reference по INT8 kernels, но **не переносится напрямую на Xtensa/RISC-V без адаптаций**. ⁵¹

- uTensor — лёгкий inference framework, ориентированный на Arm-таргеты (по описанию репозитория). Может быть интересен для обзора, но в ESP32-дипломе обычно не основной путь. ⁵²

Таблица сравнения (акцент: пригодность к ESP32 и распределённости)

Стек/ библиотека	Формат модели	ESP32 поддержка	INT8/ квантование	Оптимизации kernels	Память (модель исполнения)
TFLM / LiteRT Microcontrollers	<code>.tflite</code>	Да (в т.ч. через esp-tflite-micro)	Да (PTQ/QAT, integer-only)	Есть, зависит от портов	Требуется статическая arena
esp-tflite-micro	<code>.tflite</code>	Да (ESP-IDF интеграция)	Да	Через ESP-NN возможно ускорение	Как в TFLM

Стек/ библиотека	Формат модели	ESP32 поддержка	INT8/ квантование	Оптимизации kernels	Память (модель исполнения)
ESP-NN	kernels-библиотека	Да (ESP32/S3/ C3)	Следует спецификации TFLM INT8 kernels	SIMD/Asm на S3	Уменьшает время ops, но не отменяет arena
ESP-DL (+ ESP-PPQ)	свой формат ESP-DL	Да (ESP32-линейка)	Да (через ESP-PPQ)	Вендорный runtime	Runtime-зависим
microTVM	компиляция в C/ AOT	Частично (зависит от target/ платформы)	Возможны разные схемы	Сильная компиляция/ планирование	Может быть компактно, но зависит от модели
TinyEngine (MCUNet)	свой runtime + модели	Не “из коробки” для ESP32	Да (INT8)	memory scheduling, patch-inference	Сильно оптимизировано по SRAM
CMSIS-NN	kernels (Arm)	Нет (прямой)	Да	SIMD/MVE на Arm	N/A
uTensor	свой runtime	В основном Arm	Частично	Ориентирован на Arm	Очень лёгкий core

Источники для строки “что именно поддерживается”: репозитории и официальные описания (TFLM/LiteRT, esp-tflite-micro, ESP-NN, ESP-DL, microTVM, MCUNet/TinyEngine, CMSIS-NN, uTensor).

53

Методы оптимизации моделей для ESP32

Оптимизация для ESP32 — это совместная работа трёх уровней: **(А) алгоритм/архитектура модели**, **(В) численная компрессия (квантование/прунинг/дистилляция)**, **(С) системная реализация (память/кэш/планирование/векторизация)**. 54

Квантование: 8/4/2 бита — что реально работает на ESP32

INT8 (8-бит) — де-факто стандарт для MCU-инференса: - Схемы integer-only quantization широко приняты; классическая работа Jacob et al. описывает тренировочную процедуру, сохраняющую точность при целочисленном инференсе. 55

- В экосистеме TensorFlow есть полноценные гайды по quantization-aware training (QAT). 56

4-бит и ниже: - Исследования показывают, что “меньше бит” не всегда означает “лучше”; эффекты зависят от слоёв и задачи, и могут требовать смешанной точности и специальной поддержки kernels. 57

- На практике для ESP32 чаще всего “железобетонно” воспроизводим именно INT8, а 4/2-бит — это либо исследовательская часть (с собственными kernels), либо перенос на специализированные библиотеки/компиляторы. 58

Прунинг: почему “разреженность” часто не ускоряет MCU

Ключевая ловушка: **unstructured pruning** даёт матрицы/веса с нулями, но ускорение на MCU обычно требует либо специальных sparse-kernels, либо структурной разреженности (filters/channels), иначе “считаем нули” и теряем на адресации. Это прямо подтверждается в работах по KWS на MCU: структурный pruning более “дружелюбен” к ускорению, чем неструктурный. ⁵⁹

Дистилляция: самый “безопасный” способ получить качество при малом размере

Knowledge Distillation (Hinton et al.) — метод переноса поведения teacher-модели в компактный student, что особенно полезно для кластеров ESP32, где каждый узел может нести небольшой специализированный блок. ⁶⁰

В распределённом сценарии появляется естественная новизна: **communication-aware distillation**, когда loss включает штраф за трафик/частоту сообщений. Такая формулировка прямо согласуется с идеями NoNN (memory- и communication-aware декомпозиция teacher). ⁶¹

Компиляция/операторные оптимизации/векторизация

ESP32-S3 SIMD и ESP-NN

- В даташите ESP32-S3 явно указаны SIMD-инструкции и 128-битная data-шина. ⁶²
- ESP-NN прямо заявляет assembly-оптимизации под vector instructions ESP32-S3 и совместимость с TFLite Micro. ⁶³

Memory scheduling и “peak SRAM”

- MCUNet и MCUNetV2 показывают, что для MCU основная боль — peak SRAM, и что техники вроде patch-based inference могут снижать пик памяти в разы. ⁶⁴
- MEMA подчёркивает, что внешние обращения к памяти (и, шире, memory access patterns) критичны для энергии и производительности TinyML. ⁶⁵

Практическая рекомендация для ESP32+PSRAM: даже если модель “влезла” за счёт PSRAM, она может стать **медленнее и менее детерминированной**, поэтому измеряйте не только p50, но и p95/p99 latency. ⁶⁶

Практический чек-лист оптимизаций под дипломный прототип

- 1) Начать с INT8 PTQ/QAT и добиться baseline-качества. ⁶⁷
- 2) Ограничить операторный набор до того, что ускоряется ESP-NN/библиотеками (избегать экзотических ops без kernels). ⁶⁸
- 3) Вынести “горячие” буферы в internal SRAM, PSRAM — только для cold-данных, и измерять влияние на latency/jitter. ²¹
- 4) Включить системные метрики: p50/p95/p99, энергия на инференс/событие, throughput, packet loss. ⁶⁹
- 5) Если вы всё же делаете layer-split, использовать bottleneck-представления (BottleNet/BottleFit), иначе коммуникация “съест” выигрыш. ⁷⁰

Архитектуры распределённых нейросетей на ресурс-ограниченных узлах

Ниже — систематизация основных подходов и их применимость к ESP32-кластерам (Wi-Fi/ESP-NOW/BLE).

Основные паттерны распределения

Data parallelism (параллелизм по данным)

Каждый узел выполняет *одну и ту же модель* на разных данных. Для инференса это полезно для throughput, но не решает “один запрос быстрее”. На ESP32 применимо в задачах потоковой обработки (несколько сенсоров/несколько потоков), но редко даёт красивую демонстрацию “распределённой нейронной сети” в смысле совместного вычисления одного вывода. ⁷¹

Model parallelism (параллелизм по модели)

Модель разделяется между узлами. Классический риск: нужно передавать промежуточные активации, которые могут быть килобайтами. Это прямо является bottleneck в split-computing литературе и в обзорах по DNN partition. ⁷²

Split computing / layer-wise split

Часть слоёв на одном узле, часть — на другом. В классических работах (Edgent, Scission) это часто device-edge сценарий. Для ESP32-mesh это возможно, но без компрессии/блоков связи обычно проигрывает по latency из-за передачи features. ⁷³

Ensemble / decision-level fusion (ансамбль слияния решений)

Каждый узел имеет свою маленькую модель, отправляет логиты/оценку уверенности и метрики качества канала, а координатор делает fusion. Для ESP32 это один из самых практичных паттернов, потому что сообщение маленькое и jitter менее разрушителен. Логика “компрессии знаний в ансамбль/специалистов” концептуально перекликается с distillation и идеями specialists у Hinton et al. ⁷⁴

Mixture of Experts (MoE) / conditional compute

Запрос направляется только к “нужному” эксперту, что экономит compute и коммуникацию. Это хорошо сочетается с ESP-кластером, если routing-токен мал (десятки байт). Идеи близки к “специалистам” и к адаптивному right-sizing в Edgent. ⁷⁵

Pipeline parallelism (конвейер)

Каждый узел — стадия, запросы “текут” по конвейеру. Для ESP32 возможно только при малых тензорах между стадиями и стабильном канале, иначе увеличивается tail-latency. ⁷⁶

Trade-offs именно для ESP32: latency, bandwidth, energy, accuracy

Системная модель, полезная для диплома:

- **Latency:** $T_{e2e} \approx \sum T_{compute} + \sum T_{queue} + \sum T_{tx/rx}$. Практически T_{queue} и $T_{tx/rx}$ становятся доминирующими при Wi-Fi power-save и при нагруженном стеке. ⁷⁷

- **Bandwidth:** split computing упирается в размеры промежуточных features; потому возник целый класс работ по bottleneck-инъекции (BottleNet/BottleFit) и компрессии intermediate representations. ⁷⁰

- **Energy:** радио-передача имеет высокую цену, поэтому “передали меньше байт” часто важнее,

чем “посчитали меньше MAC”. Именно поэтому MLPerf Tiny измеряет энергию вместе с latency и accuracy. ⁷⁸

Мини-диаграммы архитектур (для вставки в диплом)

```
flowchart LR
    subgraph A["Decision-level fusion (рекомендуемо для ESP32)"]
        n1["Node 1: tiny model"] -->|"logits + conf + rssi"| c1["Coordinator: fusion"]
        n2["Node 2: tiny model"] -->|"logits + conf + rssi"| c1
        n3["Node 3: tiny model"] -->|"logits + conf + rssi"| c1
        c1 --> y["Final decision"]
    end
    end
```

```
flowchart LR
    subgraph B["Naive layer-split (риск по bandwidth)"]
        s["Sensor + early layers"] -->|"intermediate activation (KBs)"| t["tail layers on another node"]
        t --> y["Decision"]
    end
    end
```

Логика выбора первого паттерна (decision-fusion) вместо второго (naive layer-split) напрямую поддерживается наблюдениями split-computing литературы и ограничениями ESP-NOW payload/latency. ⁷⁹

Идеи для диплома и анализ “блочной” концепции configurable network

Ниже — минимум 6 вариантов дипломной работы. Во всех вариантах предполагается бюджетный стенд (3–6 плат ESP32, без дорогих роботов/дронов), а предметная задача — выбирается по доступности данных (аудио/IMU/аномалии). “Не указано”: точный дедлайн, бюджет, выбранный домен — поэтому сложность дана относительно.

Вариант А: Communication-Constrained Neural Block Graph на ESP32-кластере (рекомендуемый основной)

Формулировка задачи. Построить распределённую систему инференса, где каждый ESP32 выполняет один/несколько нейроблоков, но **межузловой обмен ограничен фиксированным бюджетом токена** (десятки байт), а маршрут вычисления выбирается адаптивно по задержке/уверенности/состоянию канала.

Мотивация. Наивный layer-split не проходит по latency из-за передачи активаций; bottleneck-представления и communication-aware дизайн — активная научная ось split computing.

⁸⁰

Архитектура (схема).

```

flowchart LR
    s["Sensor window"] --> nA["Node A: Block A + bottleneck encoder"]
    s --> nB["Node B: Block B + bottleneck encoder"]
    s --> nC["Node C: Block C + bottleneck encoder"]
    nA -->|"token (<=64B)"| R["Coordinator: routing + fusion"]
    nB -->|"token (<=64B)"| R
    nC -->|"token (<=64B)"| R
    R -->|"optional request"| nH["Heavy node: Stage-2"]
    nH -->|"refined logits"| R
    R --> y["Decision + log"]

```

Требования к узлам.

- Узлы: минимум 3× ESP32-S3 (желательно одинаковые платы для воспроизводимости). ⁸¹
- Радио: ESP-NOW для low-overhead, либо обычный Wi-Fi UDP для экспериментов throughput. ⁸²

Сложность реализации. Средняя/высокая: нужно и ML-часть (обучение блоков, дистилляция), и системную часть (protocol, синхронизация, измерения). При этом риск управляем, т.к. можно показать результат уже на decision-level fusion и постепенно усложнять.

План экспериментов.

- Baseline-1: single-node tiny model.
- Baseline-2: статический coalition-fusion (логиты) без routing.
- Baseline-3: наивный layer-split (контрольный “плохой” baseline).
- Proposed: CC-NBG + routing + stage-2.

Метрики. latency p50/p95/p99; accuracy/F1; throughput; energy per inference; traffic bytes/sec; packet loss sensitivity. ⁶⁹

Данные и бытовой прототип.

- Аудио KWS: Speech Commands (дешёвый и понятный стенд). ⁸³
- Вторая демонстрация (для “универсальности”): IMU HAR (UCI HAR) или вибро-аномалии. ⁸⁴

Вариант В: “Честный” split computing между ESP32 с обучаемым bottleneck (BottleNet-style) и сравнением жизнеспособности

Формулировка задачи. Разрезать одну модель между двумя ESP32 и показать границу, когда split выгоден/невыгоден, сравнив: (1) без компрессии features, (2) с bottleneck-кодером (BottleNet/BottleFit-подобная логика), (3) с decision-fusion. ⁷⁰

Мотивация. Split computing “упирается” в передачу intermediate features; BottleNet/BottleFit — центральные работы по сжатию features с контролем качества. ⁸⁵

Архитектура (схема).

```

sequenceDiagram
    participant U as Node 1 (head)
    participant V as Node 2 (tail)

```

```
U->>U: layers 1..k
U->>U: bottleneck encode
U->>V: token/features (compressed)
V->>V: decode (optional) + layers k+1..end
V-->>U: logits (optional) / result
```

Требования к узлам. 2× ESP32-S3 + измерение канала (RSSI) + инструментирование времени (esp_timer_get_time). ⁸⁶

Сложность реализации. Высокая (особенно если делать “настоящую” компрессию внутри модели), но научно очень чисто: вы показываете фазовую диаграмму “payload vs effective rate” и “accuracy vs compression vs latency”. ⁸⁷

Прототип. Простая аудио-классификация или ESC-50 (5-сек клипы, можно упростить до событий). ⁸⁸

Вариант С: Распределённый ансамбль специалистов (Mixture-of-Experts на ESP32) с routing по confidence и состоянию канала

Формулировка задачи. Несколько узлов обучаются как “специалисты” разных подзадач/условий (шум, разные типы сенсоров, разные комнаты), а routing выбирает, кого “спросить” и кого игнорировать, чтобы минимизировать latency/traffic. ⁷⁵

Мотивация. У Hinton et al. обсуждаются specialist-модели как альтернатива ансамблю; Edgent показывает полезность адаптивного выбора вычислений под ограничения. ⁷⁵

Схема.

```
flowchart LR
    x["Input window"] --> g["Gating: confidence + rssi"]
    g -->|select| e1["Expert 1"]
    g -->|select| e2["Expert 2"]
    g -->|select| e3["Expert 3"]
    e1 --> f["Fusion"]
    e2 --> f
    e3 --> f
    f --> y["Decision"]
```

Сложность. Средняя (сильно проще, чем “layer split”), хорошо демонстрируется на комиссии.

Датасеты. Speech Commands + синтетическое шумовое домешивание; либо IMU HAR. ⁸⁹

Вариант D: Fault-tolerant distributed inference на ESP32 (устойчивость к выпадению узлов/пакетов)

Формулировка задачи. Построить распределённую систему, где при потере части узлов/связей качество деградирует мягко; сравнить простое fusion vs “резервные связи/skip-пути” в стиле deepFogGuard. ⁹⁰

Мотивация. В распределённых DNN отказ узла ломает inference; deepFogGuard формализует failure-resilience через “skip hyperconnections” и обучение под отказы. ⁹¹

Схема.

```
flowchart LR
    A["Node A"] --> C["Coordinator"]
    B["Node B"] --> C
    D["Node D"] --> C
    A -.backup.-> C
    B -.backup.-> C
    D -.backup.-> C
```

Сложность. Средняя/высокая (нужно моделировать отказы и доказать устойчивость статистикой).

Прототип. Мультимодальный “домашний” сценарий (звук + вибрация): при отказе микрофона остаётся вибрация и наоборот. ⁹²

Вариант E: Исследовательский стенд “Split-Learning TinyML” на ESP32-S3 с измерением over-the-air протоколов

Формулировка задачи. Воссоздать/расширить постановку, где часть модели выполняется на сильно ограниченном узле, часть — на компаньоне, и сравнить транспорт (ESP-NOW vs Wi-Fi UDP vs BLE) по влиянию на latency/энергию. ⁹³

Мотивация. Тема split learning на реально ultra-low-power узлах и низкоуровневых протоколах “недоисследована” и актуальна; есть свежие работы, которые строят тестбед на ESP32-S3. ⁹⁴

Сложность. Высокая, но можно сделать как “сильный обзор + воспроизводимая инженерная репликация”, что иногда хорошо принимается на защите.

Вариант F: Distributed anomaly detection (аудио/вибрация) с кооперативной политикой “запрашивать second opinion”

Формулировка задачи. Узлы размещаются в разных точках (например, вокруг бытового прибора или в комнате), каждый классифицирует “норма/аномалия”; если уверенность низкая, запрашивается короткий токен “second stage” у другого узла/координатора. ⁹⁵

Данные. MIMII (для честной научной части) + домашний набор записей (для демонстрации). ⁹⁶

Сложность. Средняя; хорошо демонстрируется без дорогого оборудования.

Особое внимание: идея “каждый ESP32 содержит составной блок сети, суммарно конфигурируемая сеть”

Эта идея — по сути **“распределённая вычислительная ткань / neural fabric”**, где сеть собирается как граф блоков, распределённых по узлам. Сильные и слабые стороны:

Плюсы - Прямое соответствие теме диплома: распределённая сеть нейронных сетей.

- Научная ось: задача размещения блоков (placement) + выбор маршрута (routing) + компрессия межблоковых представлений. ⁹⁷

- Естественная fault tolerance: при правильной постановке можно переключать маршрут при деградации связи/отказах. ⁹¹

Минусы/риски - Если блоки соединены “как в обычной CNN” и между ними гоняются большие активации, система упирается в радио. Это фундаментальная проблема split computing. ⁹⁸

- Сложность отладки растёт нелинейно с числом узлов (тайминги, синхронизация окон, дубликаты/потери пакетов, очереди Wi-Fi task). ⁹⁹

Когда идея становится жизнеспособной (рекомендованная формулировка) - Разрезы делаются **поздно** или через **bottleneck-адаптеры** (учим маленький токен вместо сырой активации). ⁷⁰

- Маршрут вычисления **не фиксирован**, а выбирается по policy (confidence/latency/channel). ¹⁰⁰

Варианты реализации конфигурируемой сети 1) **Граф блоков с фиксированным “token budget” на ребро** (например 32–128 B). Это фактически превращает deep model в сеть “компактных сообщений”, что совместимо с ESP-NOW. ¹⁰¹

2) **Двухуровневый инференс**: always-on дешёвый путь + optional heavy path. Этот паттерн хорошо объясняется на защите и соответствует Edgent-логике right-sizing. ¹⁰²

3) **Routing как rule-based policy** (надежнее для ВКР) и как “contextual bandit” (как потенциальная новизна).

Алгоритмы маршрутизации/конфигурации (что можно написать в дипломе формально) -

Offline размещение блоков: вариация multi-constraint placement (ограничения SRAM/Flash/PSRAM каждого узла + оценка compute). В дипломе это можно делать жадным эвристическим поиском или ILP-постановкой для малых N. ¹⁰³

- Online routing: минимизация utility $U = \Delta Accuracy - \eta_1 T_{p95} - \eta_2 Energy - \eta_3 Traffic$, с измерением состояния канала и confidence. Это хорошо укладывается в литературу co-inference.

¹⁰⁴

Практические аспекты: протоколы, сериализация, синхронизация, ОТА

Выбор сетевого протокола под диплом

ESP-NOW - Плюсы: не нужен роутер/АП, низкий overhead, прямое устройство-устройство. ¹⁰⁵

- Ограничения: payload v1 250 B, v2 1470 B; “слишком короткий интервал отправки” может приводить к disorder колбэков; колбэк выполняется в high-priority Wi-Fi task — нельзя делать тяжёлую работу внутри. ¹⁰⁶

Обычный Wi-Fi (UDP/TCP) - Плюсы: выше throughput (десятки Mbit/s), проще отлаживать с iperf. ²⁴

- Минусы: больше overhead, сложнее добиться детерминированной latency, особенно при power-save. ¹⁰⁷

BLE Mesh - Плюсы: mesh-топология и механизмы provisioning/relay/low power. ¹⁰⁸

- Минусы: пропускная способность обычно ниже Wi-Fi; для передачи тензоров/частых сообщений может быть узким местом.

MQTT - Прекрасен как “прикладной протокол телеметрии/обновлений” через брокер, но для low-latency intra-cluster чаще уступает прямому ESP-NOW/UDP (из-за брокера, QoS, keepalive). В ESP-IDF есть официальный MQTT-клиент ESP-MQTT, поддерживающий MQTT v5. ¹⁰⁹

Формат обмена и сериализация тензоров

Для ESP32-кластера важно различать два типа сообщений:

- 1) **Короткие токены (логиты/латенты/confidence/качество канала)** — идеальный случай; сообщение 20–200 байт.
- 2) **Большие тензоры (активации)** — использовать только как исследовательский baseline и в поздних слоях.

Рекомендуемый подход для прототипа: фиксированный бинарный пакет (struct) без тяжёлой сериализации; при необходимости версионирования — CBOR или nanorb.

- CBOR (RFC 8949) проектировался с целью малого кода и небольших сообщений. ¹¹⁰
- nanorb — реализация Protocol Buffers с малым code size, ориентирована на MCU. ¹¹¹

Пример практического пакета ESP-NOW (из вашего документа): поля `node_id`, `seq`, локальный timestamp, квантованные logits, confidence, SNR-bin, CRC; суммарно порядка 24–32 байт.

Синхронизация времени и “fusion window”

Для multi-sensor инференса важно не абсолютное время, а согласование окон (например, аудио-окно 20–100 ms). В вашем документе предложен практический метод: периодический beacon координатора, коррекция сдвига часов, и допуск fusion-окна порядка десятков миллисекунд.

Для реализации измерений в ESP-IDF полезен `esp_timer` и система времени с микросекундным разрешением (для таймстампов событий). ¹¹²

Fault tolerance: потери пакетов и выпадение узлов

Минимально необходимое для диплома (даже без “глубокой” теории): - sequence number + idempotent пакеты, чтобы переживать дубликаты/потери;

- таймауты и ограниченные ретраи;
- fallback-режим: если узел пропал, координатор делает решение по оставшимся, снижая уверенность.

Это согласуется с общими принципами failure-resilient distributed inference, где архитектура должна быть обучена/спроектирована под отказы (deepFogGuard-логика). ¹¹³

ОТА обновление прошивки и моделей

ESP-IDF имеет полноценный OTA механизм (обновление во время работы), и отдельный компонент `esp_https_ota` для обновления по HTTPS; в документации также описаны partition tables и схема OTA разделов. ¹¹⁴

Практический паттерн для диплома: - OTA обновляет прошивку (runtime + протокол).

- Модель (веса) хранится либо как часть firmware image, либо в отдельном разделе flash/файловой системы; обновляется контролируемо и с версионированием.

Экспериментальный план, измерения и рекомендации по прототипу

Что именно измерять (и как сделать результаты “защитимыми”)

Опирайтесь на дисциплину MLPerf Tiny: **latency + energy + accuracy** и строгая воспроизводимость. ¹¹⁵

Минимальный набор метрик - Accuracy/F1 (или FAR/FRR для KWS). ¹¹⁶

- Latency: p50/p95/p99 end-to-end (по событиям: capture → decision). ¹¹⁷
- Throughput: окна/сек или инференсов/сек, отдельно compute и с сетью. ²⁴
- Energy: энергия на инференс / на событие, а также “в час” при always-on режиме. ¹¹⁸
- Traffic: байт/сек и байт/решение; packet loss rate и влияние на качество. ¹¹⁹

Набор тестов: от микро-бенчмарков к полноценной системе

Тест 1: инференс одного блока (compute microbench)

- На каждом узле по отдельности: время блока/слоя, peak SRAM, влияние PSRAM. ¹²⁰

Тест 2: передача токенов (network microbench)

- RTT и one-way latency для сообщений 32/64/128/256/512/1024 байт (ESP-NOW и Wi-Fi UDP).
- Отдельно: jitter при включённом/выключенном power-save (WIFI_PS_NONE vs default), потому что delay может вырасти до DTIM/listen interval. ¹²¹

Тест 3: полный end-to-end пайплайн

- Single node baseline.
- Multi-node fusion baseline.
- Proposed routing/stage-2.
- Абляции: без distillation, без ESP-NN, без adaptive routing. ¹²²

Инструменты измерений

Временные метки - `esp_timer_get_time()` и системное время ESP-IDF (1 μ s resolution в стандартной конфигурации времени). ¹²³

Трассировка - Application level tracing и SystemView для анализа планировщика и очередей. ³⁸

Энергия - Идеально: внешний измеритель тока/энергии (либо шунт + АЦП/модуль измерения). В вашем документе корректно указано, что энергия считается интегралом $E = \int V \cdot I(t) dt$ и что нужно достаточно частое измерение, чтобы поймать пики радио.

Рекомендации по реализации прототипа

Минимальный набор оборудования (бюджетный) - 3–5× ESP32-S3-DevKitC-1 (или аналогичные официальные платы). ³³

- Сенсоры под домен: - аудио: I2S микрофоны (бюджетно и хорошо объяснимо на защите);
- IMU: типовой акселерометр/гироскоп модуль;
- вибрация: пьезо/акселерометр.
- (Опционально) измеритель тока или хотя бы решение “шунт + логгер”.

ПО - ESP-IDF как основа (скрипты сборки/прошивки, контроль FreeRTOS задач). ⁴⁰

- Для ML: esp-tflite-micro + ESP-NN либо ESP-DL (зависит от трубопровода модели). ¹²⁴

Рекомендуемая структура задач (важно для jitter) В вашем документе дана практическая раскладка: audio_task → feature_task → infer_task → radio_tx_task → control_task, и рекомендация не перегружать сетевые callback-и вычислениями (только очереди/буферы). Это согласуется с ESP-NOW документацией о high-priority Wi-Fi task. ¹²⁵

Таймлайн (Gantt, адаптируйте под “не указано” дедлайн)

В вашем документе предложена дорожная карта на 16 недель, которую удобно превратить в диаграмму.

```
gantt
    title План работ по ВКР
    dateFormat YYYY-MM-DD
    axisFormat %d.%m

    section Постановка и протокол
    Уточнение задачи, метрик, baselines :a1, 2026-02-17, 14d

    section Single-node baseline
    Локальный пайплайн + измерения compute :a2, after a1, 14d

    section Сеть и кооперация
    ESP-NOW протокол + 2-3 узла :a3, after a2, 14d
    Fusion v1 + сбор логов :a4, after a3, 14d

    section Оптимизация и абляции
    INT8/QAT + distillation :a5, after a4, 14d
```


ESP-NN/операторные оптимизации	:a6, after a5, 14d
section Адаптивность и устойчивость	
Adaptive routing + stage-2	:a7, after a6, 14d
Fault tolerance (потери/отказы)	:a8, after a7, 14d
section Масштабирование и финал	
4-5 узлов + полный benchmark	:a9, after a8, 14d
Текст, графики, репозиторий, демо	:a10, after a9, 14d

Источники и приложения

Аннотированный список ключевых ссылок

Ниже — наиболее релевантные источники, которые стоит прямо цитировать в дипломе (официальные доки Espressif, репозитории, статьи и современные обзоры). Переход по ссылке осуществляется через кликабельную ссылку-цитату.

Официальная документация и стек Espressif - Даташит ESP32 (CPU LX6, SRAM, периферия, power modes). ⁹

- Даташит ESP32-S3 (SIMD, SRAM 512KB, deep-sleep ток, Wi-Fi/BLE). ¹²⁶
- ESP-NOW API (payload v1/v2, ограничения callback-ов и темпа отправки). ¹⁰⁶
- Wi-Fi performance + power save (DTIM/listen interval как источник задержек). ¹²⁷
- External RAM/PSRAM guide (ограничения и практика allocation). ¹²⁸
- OTA и esp_https_ota. ¹²⁹
- FreeRTOS SMP в ESP-IDF (важно для пиннинга задач и детерминизма). ¹³⁰

Репозитории и runtime'ы - esp-tflite-micro (примеры TFLM под ESP-IDF). ¹³¹

- ESP-NN (оптимизированные kernels, S3 vector ускорение). ¹³²
- ESP-DL (inference framework + инструмент квантования ESP-PPQ). ¹³³
- MLPerf Tiny (бенчмарк дисциплины latency/energy/accuracy). ¹³⁴

Классические и опорные научные статьи - Integer-only quantization (Jacob et al.). ⁵⁵

- Knowledge Distillation (Hinton et al.). ⁶⁰
- MCUNet / MCUNetV2 (memory scheduling, patch inference). ¹³⁵
- NoNN (communication-aware distributed inference для IoT). ⁶¹
- BottleNet / BottleFit (bottleneck-компрессия features для split computing). ⁸⁵
- Edgent (dynamic co-inference + early-exit). ¹³⁶
- Обзор partition/split: survey по DNN partition. ¹³⁷
- Failure-resilience: deepFogGuard. ⁹¹
- Split-Learning TinyML на ultra-low-power узлах (свежий экспериментальный testbed, релевантно к ESP32-S3). ¹³⁸

Датасеты для дешёвых и воспроизводимых демонстраций - Speech Commands (KWS). ⁸³

- UCI HAR (IMU-активности). ⁸⁴
- ESC-50 (экологические звуки). ⁸⁸
- MIMII (звуковые аномалии машин). ⁹⁶

Приложение: минимальный пример кода TFLite Micro на ESP32-S3 (скелет)

Нижне — “канонический” минимальный скелет для ESP-IDF проекта с TFLM-подходом: статическая arena, MicroMutableOpResolver, интерпретатор, вызов `Invoke()`. Конкретные include-пути зависят от того, используете ли вы esp-tflite-micro как компонент через ESP-IDF Component Manager или как подмодуль.

```
// main/main.cpp (ESP-IDF)
// Минимальный скелет инференса TFLite Micro на ESP32

#include <stdio.h>
#include "esp_timer.h"
#include "esp_log.h"

// TFLite Micro headers (пути зависят от интерпрации)
#include "tensorflow/lite/micro/micro_interpreter.h"
#include "tensorflow/lite/micro/micro_mutable_op_resolver.h"
#include "tensorflow/lite/schema/schema_generated.h"
#include "tensorflow/lite/micro/micro_log.h"

// Модель (как C array), сгенерированная из .tflite
extern const unsigned char g_model[];
extern const int g_model_len;

static const char* TAG = "tflm-demo";

// Подберите размер arena экспериментально (начните с 64-256KB на ESP32-S3)
constexpr int kTensorArenaSize = 128 * 1024;
alignas(16) static uint8_t tensor_arena[kTensorArenaSize];

extern "C" void app_main(void) {
    ESP_LOGI(TAG, "Boot. Starting TFLM...");

    const tflite::Model* model = tflite::GetModel(g_model);
    if (model->version() != TFLITE_SCHEMA_VERSION) {
        ESP_LOGE(TAG, "Model schema mismatch: %d != %d",
                  model->version(), TFLITE_SCHEMA_VERSION);
        return;
    }

    // Разрешаем только нужные ops (уменьшает размер бинарника и памяти)
    tflite::MicroMutableOpResolver<8> resolver;
    resolver.AddFullyConnected();
    resolver.AddConv2D();
    resolver.AddDepthwiseConv2D();
    resolver.AddAveragePool2D();
    resolver.AddReshape();
    resolver.AddSoftmax();
    resolver.AddMul();
    resolver.AddAdd();
```

```

    tflite::MicroInterpreter interpreter(model, resolver, tensor_arena,
kTensorArenaSize);

    if (interpreter.AllocateTensors() != kTfLiteOk) {
        ESP_LOGE(TAG, "AllocateTensors failed. Increase arena?");
        return;
    }

    TfLiteTensor* input = interpreter.input(0);
    TfLiteTensor* output = interpreter.output(0);

    // TODO: заполнить input->data.int8 / input->data.f (в зависимости от
модели)

    int64_t t0 = esp_timer_get_time();
    TfLiteStatus st = interpreter.Invoke();
    int64_t t1 = esp_timer_get_time();

    if (st != kTfLiteOk) {
        ESP_LOGE(TAG, "Invoke failed");
        return;
    }

    ESP_LOGI(TAG, "Invoke time: %lld us", (long long)(t1 - t0));

    // TODO: прочитать output->data.int8 / output->data.f
}

```

Смысл ключевых элементов: - “Memory arena” — центральная идея TFLM: все intermediate tensors размещаются внутри заданного буфера (что делает footprint предсказуемым). ¹³⁹

- `MicroMutableOpResolver` позволяет линковать только нужные операторы (что помогает контролировать размер бинарника). ¹⁴⁰

- На Espressif практический старт проще всего сделать через esp-tflite-micro примеры. ¹³¹

Приложение: пример конфигурации распределённой сети (псевдо-YAML)

Пример “конфигурируемой сети” как графа блоков (под вариант A):

```

network:
  token_budget_bytes: 64
  coordinator:
    node_id: 0
  policy:
    type: rule_based
    tau_low_entropy: 0.35
    tau_high_entropy: 0.65
    require_min_rssi_dbm: -75
  nodes:
    - node_id: 1

```

```

    role: sensor+expert
    model: kws_student_v1_int8
    sends:
      - type: logits_q7
      - type: confidence_u8
      - type: rssi_snr
  - node_id: 2
    role: sensor+expert
    model: kws_student_noise_robust_int8
  - node_id: 3
    role: heavy_stage2
    model: kws_stage2_int8
transport:
  type: esp_now
  packet:
    fields: [node_id, seq, ts_local, logits_q7, conf, snr_bin, crc16]

```

Поля пакета соответствуют практическому шаблону (node_id/seq/timestamp/logits/conf/snr/crc), а routing-пороговые параметры отражают идею “local-only vs fusion vs stage-2”. ¹⁴¹

Приложение: протокол обмена (sequence) и требования к колбэкам

```

sequenceDiagram
    participant Ni as Node i
    participant WiFi as Wi-Fi / ESP-NOW stack
    participant C as Coordinator
    Ni->>Ni: capture + features + infer
    Ni->>WiFi: esp_now_send(packet)
    Note over WiFi: callback в high-priority Wi-Fi task
    WiFi->>C: packet RX
    C->>C: fusion + routing decision
    C-->>Ni: optional request (stage-2)

```

Критично, что логика ESP-NOW предписывает не делать тяжёлую работу внутри send callback и учитывать ограничения при частой отправке. ¹⁰⁶

Приложение: шаблон таблицы итоговых результатов (для главы “Эксперименты”)

Сценарий	Архитектура	Payload/ решение	p50 latency	p95 latency	p99 latency	Accuracy/ F1	Energy/ решение	Комментарий
Baseline-1	single node	0 B	...	...	...	...	...	локальная модель
Baseline-2	fusion logits	~32-128 B	...	...	...	...	...	кооперация без stage-2
Baseline-3	naive layer-split	KBs	...	...	...	...	...	демонстрация bottleneck

Сценарий	Архитектура	Payload/ решение	p50 latency	p95 latency	p99 latency	Accuracy/ F1	Energy/ решение	Комментарий
Proposed	CC-NBG + routing	~32~128 В (+ редко stage-2)	...	...	...	...	...	экономия по p95/p99

Такой формат прямо поддерживает требование MLPerf Tiny измерять latency/energy/accuracy вместе и позволяет показать выигрыш по tail-latency, что для Wi-Fi power-save и jitter особенно важно. ¹⁴²

¹ ⁵ ⁶ ⁶¹ ⁹⁷ <https://arxiv.org/abs/1907.11804>

<https://arxiv.org/abs/1907.11804>

² ¹⁴ ¹⁵ ¹⁷ ¹⁸ ²² ²⁸ ⁶² ⁸⁶ ¹²⁶ documentation.espressif.com

https://documentation.espressif.com/api/resource/doc/file/rz94aWY3/FILE/esp32-s3_datasheet_en.pdf

³ ⁸² ⁹⁹ ¹⁰¹ ¹⁰⁶ ¹¹⁹ ¹²⁵ ¹⁴¹ [ESP-NOW - ESP32 - — ESP-IDF Programming Guide v5. ...](https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html?utm_source=chatgpt.com)

https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/network/esp_now.html?utm_source=chatgpt.com

⁴ ⁷³ ¹⁰⁰ ¹⁰² ¹⁰⁴ ¹³⁶ <https://arxiv.org/abs/1806.07840>

<https://arxiv.org/abs/1806.07840>

⁷ ¹⁰ [ESP32 Datasheet](https://documentation.espressif.com/esp32_datasheet_en.html?utm_source=chatgpt.com)

https://documentation.espressif.com/esp32_datasheet_en.html?utm_source=chatgpt.com

⁸ ²⁵ ⁷⁷ ¹⁰⁷ ¹¹⁷ ¹²¹ [Wi-Fi Performance and Power Save - ESP32](https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/wifi-driver/wifi-performance-and-power-save.html?utm_source=chatgpt.com)

https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/wifi-driver/wifi-performance-and-power-save.html?utm_source=chatgpt.com

⁹ ²³ ²⁶ [ESP32 Datasheet](https://documentation.espressif.com/esp32_datasheet_en.html)

https://documentation.espressif.com/esp32_datasheet_en.html

¹¹ [ESP32-S2 Series](https://espressif.com/sites/default/files/esp32-s2-v1.0_datasheet_en.pdf?utm_source=chatgpt.com)

https://espressif.com/sites/default/files/esp32-s2-v1.0_datasheet_en.pdf?utm_source=chatgpt.com

¹² ¹³ [ESP32-C3 Wi-Fi & BLE 5 SoC](https://www.espressif.com/en/products/socs/esp32-c3?utm_source=chatgpt.com)

https://www.espressif.com/en/products/socs/esp32-c3?utm_source=chatgpt.com

¹⁶ [Configure ESP32-S3 Built-in JTAG Interface](https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/jtag-debugging/configure-builtin-jtag.html?utm_source=chatgpt.com)

https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/jtag-debugging/configure-builtin-jtag.html?utm_source=chatgpt.com

¹⁹ ¹³⁹ <https://www.coral.ai/docs/reference/micro/tensorflow/>

<https://www.coral.ai/docs/reference/micro/tensorflow/>

²⁰ ²¹ ⁶⁶ ¹²⁰ ¹²⁸ [Support for External RAM - ESP32-S3](https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/external-ram.html?utm_source=chatgpt.com)

https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/external-ram.html?utm_source=chatgpt.com

²⁴ ¹²⁷ <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/wifi-driver/wifi-performance-and-power-save.html>

<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/wifi-driver/wifi-performance-and-power-save.html>

²⁷ [ESP32-S3 Wi-Fi & BLE 5 SoC](https://www.espressif.com/en/products/socs/esp32-s3?utm_source=chatgpt.com)

https://www.espressif.com/en/products/socs/esp32-s3?utm_source=chatgpt.com

29 69 71 78 115 118 142 <https://arxiv.org/abs/2106.07597>
<https://arxiv.org/abs/2106.07597>

30 <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/index.html>
<https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/index.html>

31 https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s2/esp32-s2-devkitm-1/user_guide.html
https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s2/esp32-s2-devkitm-1/user_guide.html

32 <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c3/esp32-c3-devkitm-1/index.html>
<https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c3/esp32-c3-devkitm-1/index.html>

33 81 <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html>
<https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html>

34 40 <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-py.html>
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-py.html>

35 <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-monitor.html>
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-monitor.html>

36 <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/partition-tables.html>
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/partition-tables.html>

37 JTAG Debugging: ESP32-S3 - ESP-IDF Programming Guide
[https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/jtag-debugging/index.html?](https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/jtag-debugging/index.html?utm_source=chatgpt.com)
[utm_source=chatgpt.com](https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/jtag-debugging/index.html?utm_source=chatgpt.com)

38 https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/app_trace.html
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/app_trace.html

39 Getting Started - - — Arduino ESP32 latest documentation
https://docs.espressif.com/projects/arduino-esp32/en/latest/getting_started.html?utm_source=chatgpt.com

41 micropython/ports/esp32/README.md at master
https://github.com/micropython/micropython/blob/master/ports/esp32/README.md?utm_source=chatgpt.com

42 55 67 <https://arxiv.org/abs/1712.05877>
<https://arxiv.org/abs/1712.05877>

43 53 https://ai.google.dev/edge/litert/microcontrollers/get_started
https://ai.google.dev/edge/litert/microcontrollers/get_started

44 140 <https://github.com/tensorflow/tflite-micro>
<https://github.com/tensorflow/tflite-micro>

45 124 131 <https://github.com/espressif/esp-tflite-micro>
<https://github.com/espressif/esp-tflite-micro>

46 63 68 122 132 <https://github.com/espressif/esp-nn>
<https://github.com/espressif/esp-nn>

47 133 <https://github.com/espressif/esp-dl>
<https://github.com/espressif/esp-dl>

48 https://daobook.github.io/tvm/docs/arch/microtvm_design.html
https://daobook.github.io/tvm/docs/arch/microtvm_design.html

49 54 64 135 <https://arxiv.org/abs/2007.10319>
<https://arxiv.org/abs/2007.10319>

50 <https://github.com/mit-han-lab/tinyengine>
<https://github.com/mit-han-lab/tinyengine>

51 <https://github.com/ARM-software/CMSIS-NN>
<https://github.com/ARM-software/CMSIS-NN>

52 <https://github.com/uTensor/uTensor>
<https://github.com/uTensor/uTensor>

56 https://www.tensorflow.org/model_optimization/guide/quantization/training
https://www.tensorflow.org/model_optimization/guide/quantization/training

57 58 <https://arxiv.org/abs/2203.05492>
<https://arxiv.org/abs/2203.05492>

59 <https://arxiv.org/abs/2208.02765>
<https://arxiv.org/abs/2208.02765>

60 74 75 <https://arxiv.org/abs/1503.02531>
<https://arxiv.org/abs/1503.02531>

65 <https://arxiv.org/abs/2304.05544>
<https://arxiv.org/abs/2304.05544>

70 85 <https://arxiv.org/abs/1902.01000>
<https://arxiv.org/abs/1902.01000>

72 76 80 87 98 137 <https://arxiv.org/pdf/2304.10020>
<https://arxiv.org/pdf/2304.10020>

79 <https://arxiv.org/abs/2201.02693>
<https://arxiv.org/abs/2201.02693>

83 89 116 <https://arxiv.org/abs/1804.03209>
<https://arxiv.org/abs/1804.03209>

84 <https://archive.ics.uci.edu/ml/datasets/human%2Bactivity%2Brecognition%2Busing%2Bsmartphones>
<https://archive.ics.uci.edu/ml/datasets/human%2Bactivity%2Brecognition%2Busing%2Bsmartphones>

88 <https://www.karolpiczak.com/papers/Piczak2015-ESC-Dataset.pdf>
<https://www.karolpiczak.com/papers/Piczak2015-ESC-Dataset.pdf>

90 91 113 <https://arxiv.org/abs/1909.00995>
<https://arxiv.org/abs/1909.00995>

92 95 96 <https://arxiv.org/abs/1909.09347>
<https://arxiv.org/abs/1909.09347>

93 94 138 <https://arxiv.org/abs/2507.16594>
<https://arxiv.org/abs/2507.16594>

103 <https://paul-harvey.org/publication/scission/scission.pdf>
<https://paul-harvey.org/publication/scission/scission.pdf>

105 <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnw.html>
<https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnw.html>

108 <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/esp-ble-mesh/ble-mesh-index.html>
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/esp-ble-mesh/ble-mesh-index.html>

109 <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html>
<https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/protocols/mqtt.html>

110 <https://www.rfc-editor.org/rfc/rfc8949.html>
<https://www.rfc-editor.org/rfc/rfc8949.html>

111 <https://github.com/nanopb/nanopb>
<https://github.com/nanopb/nanopb>

112 https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/esp_timer.html
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/esp_timer.html

114 129 Over The Air Updates (OTA) - ESP32
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/ota.html?utm_source=chatgpt.com

123 https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/system_time.html
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/system_time.html

130 <https://docs.espressif.com/projects/esp-idf/en/v5.0/esp32s3/api-guides/freertos-smp.html>
<https://docs.espressif.com/projects/esp-idf/en/v5.0/esp32s3/api-guides/freertos-smp.html>

134 <https://github.com/mlcommons/tiny>
<https://github.com/mlcommons/tiny>